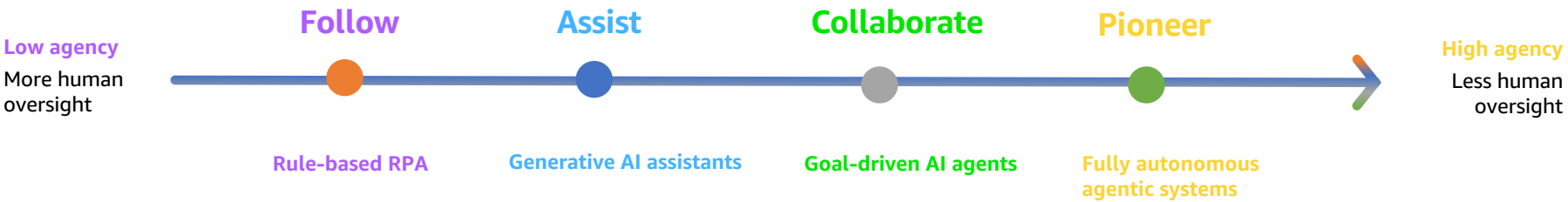
Don’t Let Your Agent Freelance Against the Warehouse

Ontologies and Progressive Disclosure

The Agency Curve

As autonomy rises, so does the criticality of semantic understanding



INCREASING Criticality ON SEMANTIC Understanding

It's not what you say, it's what you mean

Two failure modes that block agentic analytics

Grounding and delivery - same problem, two surfaces

1 The agent doesn't know what the business means

- Text-to-SQL hallucinations
- Wrong joins, wrong filters
- **“Active rider”**: in-trip now? paid in last 28 days? logged in within 90?

2 The agent drowns in context

- Every table description, every metric loaded upfront
- Real warehouses have thousands of tables
- One-shot "give the LLM the schema" is impossible

A “data ≠ understanding” moment

Analyst asks: “What’s the average pickup time at Boston Logan Airport this quarter?” Same data, one ontology rule apart.

✗ Data-only AI

```
SELECT AVG(pickup_min)
FROM   trips
WHERE  pickup_zone = 'Logan'
  AND  status = 'completed';
-- no compliance rule
```

→ 2.4 min fast, confident, wrong

Counts illegal curb pickups — $200 fines, $5.50 airport fee skipped.

✓ Knowledge-enabled AI

```
SELECT AVG(pickup_min)
FROM   trips
WHERE  pickup_zone = 'Logan'
  AND  status = 'completed'
  AND  is_compliant = TRUE  -- ontology
```

→ 8.1 min compliant, correct

One ontology rule filters to compliant trips — and surfaces “12% excluded per policy.”

The data was the same. The understanding was not.

The Thesis

Semantic layers solve **grounding.**

Progressive disclosure solve **delivery.**

You need both.

Semantic Layer

The Four Semantic Barriers

What blocks high AI agency over structured data

CONTEXT

Limited information access

“Top driver markets”
agent doesn't know market_id was deprecated.

CONNECTION

Relationship blindness

“Riders who switched payment after a low rating”
needs Trip→Rating→Rider→PaymentEvent.

REASONING

No class/type discipline

“Show all Vehicles rated >4.7”
confuses driver, rider, trip ratings.

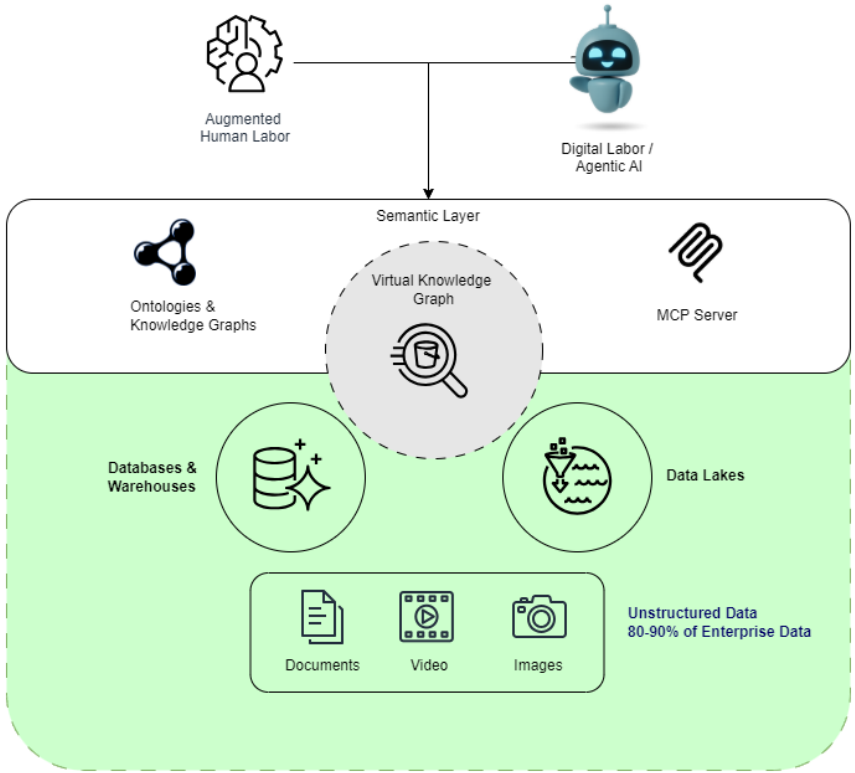
TRUST

No explainability or governance

“Why this number?”
no lineage, no policy provenance.

The Foundation to Scale Autonomous Agentic AI

A **Semantic Layer** is an abstraction layer that's grounded in ontology to enforce structure and support inferencing, and a virtual knowledge graph provides federation and representation of enterprise data and information as graphs.



What Does “Semantic” Mean? *Different Things*

WHERE YOU SEE IT	WHAT IT BUYS YOU	LIMIT
Vector embeddings	Meaning ≈ proximity in vector space	*No constraints, rules, or inference*
Knowledge graphs	Labeled nodes/edges; who connects to what	*Captures relationships but not why they're valid*
Ontologies	First-order logic + rules + constraints	*Enables inference, validation, explanation*
BI semantic models	Calculation definitions (e.g., GMV)	*Metric layer only — no graph, no inference*

Vectors → Graphs → Ontology

Why you need all three

1

Vectors find similarity

RAG over text descriptions; meaning is proximity in a learned space

Driver rating and trip rating embed close — not interchangeable

2

Graphs preserve structure

Bill livesIn Paris; Eiffel Tower locatedIn Paris

3

Ontologies enforce meaning

owns has domain Person, range Asset; City is not an Asset

Schema vs. Ontology

Shape contract vs. meaning contract

SCHEMA says…

the columns exist.

```
CREATE TABLE trip (
  trip_id BIGINT PRIMARY KEY,
  rider_id BIGINT,
  rating  DECIMAL(2,1),
  ...
);
```

ONTOLOGY says…

what they mean & what's valid.

```
• :Trip a owl:Class .
• :rating rdfs:domain :Trip ;
        rdfs:range  xsd:decimal .
• SHACL: rating ∈ [1.0, 5.0] .
• :Trip subClassOf schema:Trip .
```

SHACL = Shapes Constraint Language — W3C standard for validating RDF graphs.

Why Ontology Unlocks Each Barrier

Class/property axioms

→ fixes CONTEXT

region_id is the live identifier, not deprecated market_id.

Knowledge integration

→ fixes CONNECTION

Pricing, T&S, Finance traverse the same shared Trip class.

Reasoning & consistency

→ fixes REASONING

RidesharingTrip without serviceClass is inferred from vehicleClass; impossible states get rejected.

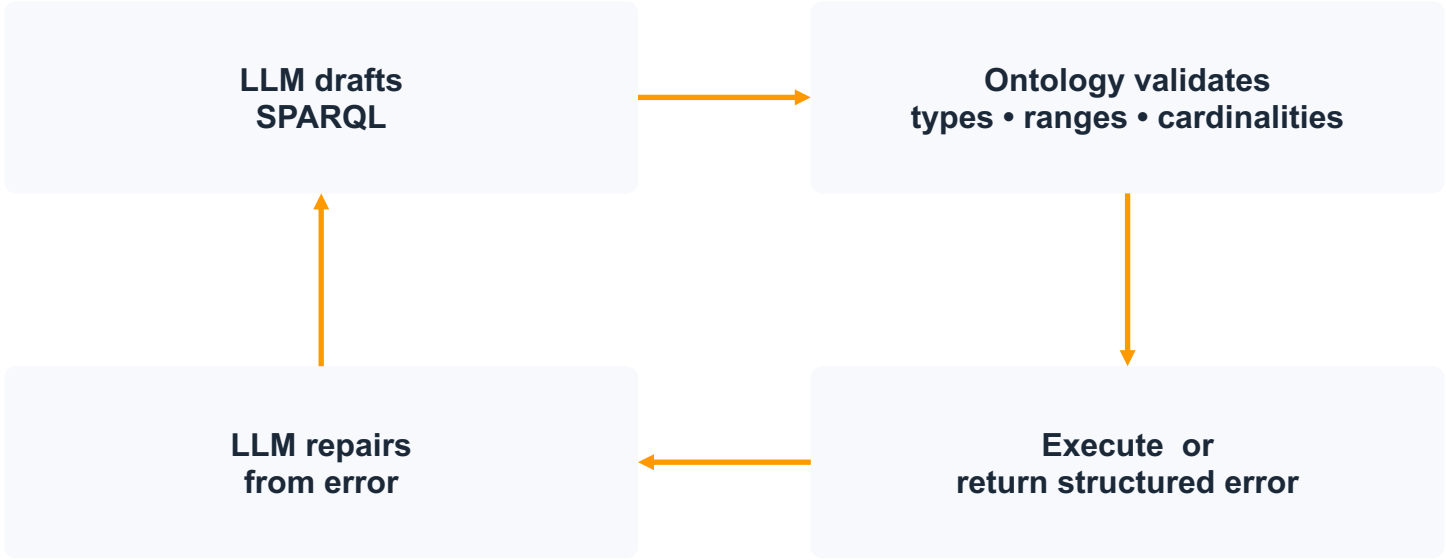
Governance & trust

→ fixes TRUST

Every answer cites its source axiom

Neuro-symbolic: LLM Proposes, Ontology Disposes

The loop that makes text-to-SQL trustworthy

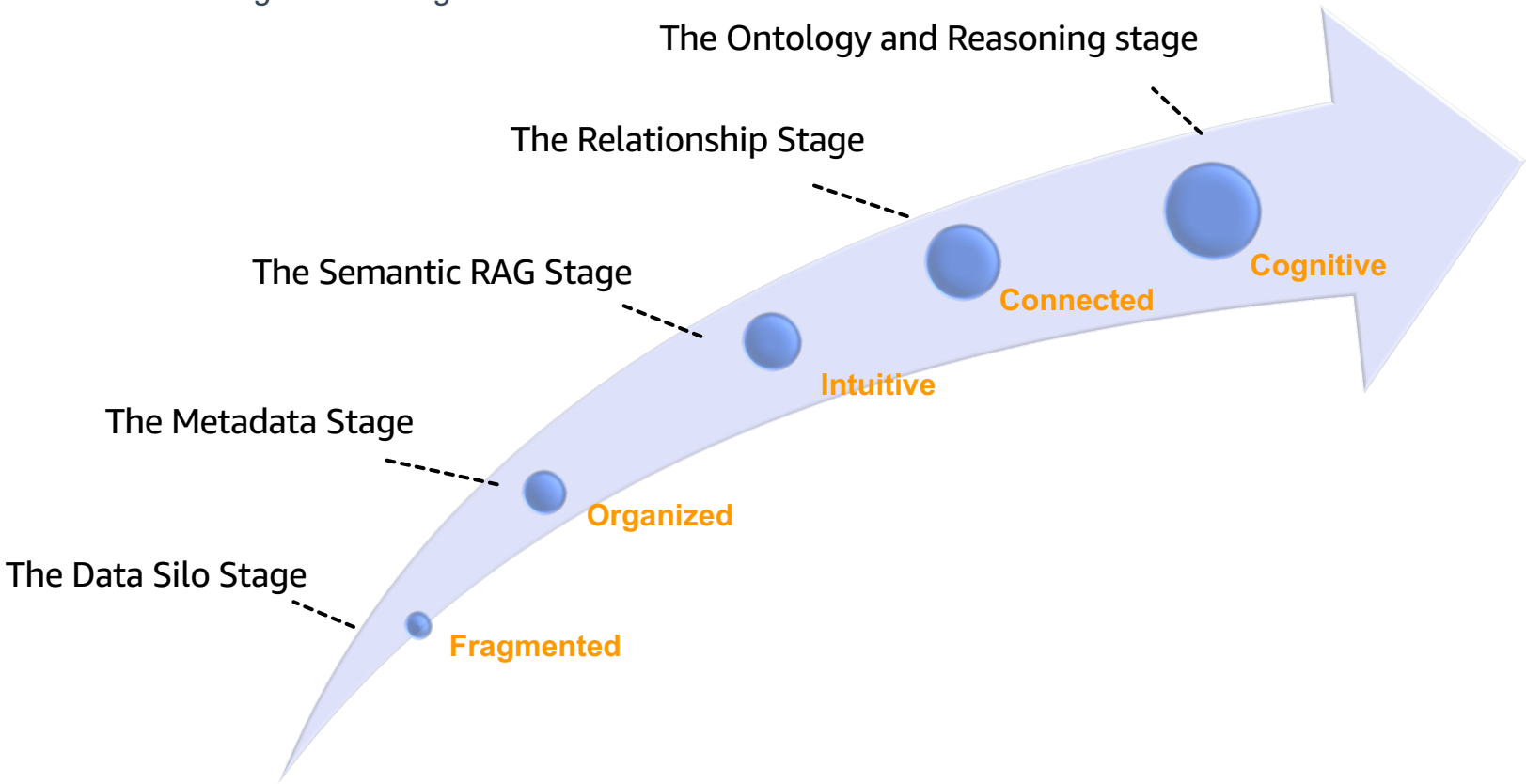


“The model gets to be creative; the ontology gets to be strict.”

A semantic layer over a bad data layer is just a fresh coat of paint on a crumbling foundation

The Semantic Maturity Curve

From Data Silos to cognitive Intelligence



Bind agents to a governed semantic layer and centralize it

Text-to-SQL accuracy

20% → **92.5%**

Raw schema → governed semantic layer
(AtScale 2024 benchmark)

- **Metric / measure layers**
- **Knowledge graphs / ontologies**
- **Hybrid context catalogs**

Architecture pattern: semantic-layer-as-MCP-tool. Bind to the API, not the prompt.

Anti-pattern: “you can write SQL” prompt + table DDL

Reuse upper ontologies: inherit, don’t re-invent

Schema.org

Trip · Vehicle · Reservation · Rating · Place

GeoSPARQL

Pickup zones · geofencing · service-area polygons

OWL-Time

ETAs · surge windows · scheduling · after-hours pricing

Anchor each local class to an external IRI — `RidesharingTrip subClassOf schema:Trip`

Keep it repeatable: LLM-assisted ontology generation + RAG over canonical ontologies + re-trigger on source change

Anti-pattern: hand-author classes from scratch · skip IRI anchors · freeze in a wiki and discover the term means something different in every domain six months later

Federate definitions; don’t flatten them

“Active rider” means three different things: Operations, Finance, Trust & Safety. Pick one and you’ve broken two.

Operations

Active rider =

in-trip right now

Finance

Active rider =

took a paid trip in the last 28 days

Trust & Safety

Active rider =

logged in within 90 days, not flagged

Pattern: **federation, not flattening.** Each domain keeps its definition; all aligned to an upper layer.

Industry references — FIBO (finance) · SNOMED (clinical) · KGS-style federations (Bloomberg, Google)

Anti-pattern: convene a 6-week working group to ratify “the” definition · ship it · watch each domain quietly maintain a shadow definition

Treat the layer as evolving infrastructure

Static semantic layers rot. The ones that ship are versioned, evaluated, and observed like application code.

Version like code

Eval on every change

Plan for evolution

Optimize from logs

Observe: context-token usage · retrieval hit rate · slice size · tier mix → OTEL from day one.

Anti-pattern: ship the ontology, freeze it, declare victory · discover a year later that 30% of definitions are stale

Two Architectural Patterns

Start pragmatic; promote when usage justifies it

(A) Semantic RAG over a proto-ontology

Vector knowledge base

Ships in weeks. Often enough.

- Per-table schema docs
- Business glossary
- Domain / role ontology
- Lookup / type-code reference

Indexed → retrieved per query.

(B) Virtual Knowledge Graph

Graph database + SPARQL

Ships in months.

- Formal axioms in OWL
- Inference + cross-domain alignment
- Data quality and business rule constraints
- Provable reasoning

VKG = relational rows mapped to RDF triples at query time — none materialized.

→ Promote (A) → (B) when query logs justify it.

Progressive Disclosure

Progressive Disclosure for Text-to-SQL

Civili et al., RelationalAI (2024) - validated on AT&T telecom KG (500 concepts, 1k relationships, 7,000+ axioms)

1

Approximation

Surface relevant concepts using NL labels only

Even thousands of concepts fit in context

2

Refinement

Slice ontology to candidate concepts; iterate with LLM

3

Translation

Generate SPARQL/SQL against the validated slice

Anti-pattern: build only Phase 3 · paste the full ontology into the prompt

Phase 1: Approximation (NL → NL)

Topic Router

Mechanics

LLM sees only NL labels of concepts/relationships from the upper ontology

Compact label index + embedding-similarity router. Cheap to run.

Example

Q: “Average surge multiplier for SF airport pickups, Friday evening rush, last quarter?”

→ {Trip, PickupZone, SurgeWindow, ServiceArea, TimeOfDay}

Phase 2: Refinement (NL → formal, iterative)

Slice Builder

Mechanics

External ontology service slices full ontology to subgraph; T-Box path-finding finds connecting paths

LLM iterates: “Can you translate? If not, what’s missing?” (typically 2–3 rounds)

Example

Slice expands to: SurgeWindow appliesTo ServiceArea; Trip occursIn ServiceArea, Trip hasFare Fare hasSurgeMultiplier

Follow-up: “How is ‘airport’ modeled?” → Airport subClassOf PointOfInterest, locatedIn ServiceArea

Phase 3: Translation (formal → SPARQL/SQL)

SPARQL/SQL Gen

Mechanics

LLM generates SPARQL (or SQL via VKG mappings) against the minimal validated slice

Validate against ontology axioms (types, ranges, cardinalities) before execution

Example

Trip filtered on pickupZone, timeOfDay, serviceArea = SF; aggregate avg(surgeMultiplier)

Mappings translate to SQL. Most public TEXT_TO_SPARQL_PROMPT examples are Phase 3 only.

Tiered query resolution: cheapest deterministic path first

A lot of questions match a governed metric. They don't need an LLM.

TIER	WHAT RUNS	LLM CALLS	RIDE-SHARE EXAMPLE
T1	Pre-determined metric lookup	**0-1**	*"GMV by region last quarter"*
T2	NL → SPARQL/SQL via 3-phase pipeline	**0–3**	*"Avg surge for SF airport Friday evening Q4"*
T3	Agentic fallback over the broader semantic layer	**many**	*"Why did driver retention drop in Austin in March?"*

Anti-pattern: every NL question gets the agentic treatment, including "GM by region last quarter," burning tokens

Disambiguate before you generate

A clarifying question is cheaper than a wrong SQL run.

> “Did you mean active rider in
>
> **Finance terms (paid, last 28d) or Trust & Safety (login, 90d)?”**

> **“GM = General Motors or gross margin?”**

WHERE TO INSERT IT

Between Phase 1 (concept set) and Phase 2 (slice build) of the 3-phase pipeline.

TRIGGER HEURISTICS

ambiguous term hits >1 federated definition · concept set spans >1 domain · low-confidence retrieval score

BONUS *every clarification answer is a labeled training signal — feed it back into the layer.*

Anti-pattern: pick the most popular definition silently · ship a wrong number to the CFO · debug the SQL instead of the question

Semantic layers + ontology give agents **truth.**

Progressive disclosure gives them **attention.**

You need both.

ADDITIONAL RESOURCES

- Paper: Using off-the-shelf LLMs to query enterprise data by progressively revealing ontologies
- Paper: Retrieval-Augmented Generation of Ontologies from Relational Databases
- AWS Sample: `github.com/aws-samples/sample-semantic-layer-structured`
- AWS Sample: `github.com/aws-samples/sample-semantic-layer-using-agents`